

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

**«ЗАПРОСЫ НА ВЫБОРКУ И МОДИФИКАЦИЮ ДАННЫХ.
ПРЕДСТАВЛЕНИЯ. РАБОТА С ИНДЕКСАМИ»**

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Янин Егор Вячеславович

Факультет ФПИИ

Группа К3241

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2024

Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025/2026

Цель работы

Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Выполнение

Практическое задание 2.1.1:

1. *Создайте базу данных `learn`.*
2. *Заполните коллекцию единорогов `unicorns`*
3. *Используя второй способ, вставьте в коллекцию единорогов документ:*

```
{name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm',  
vampires: 165}
```

4. *Проверьте содержимое коллекции с помощью метода `find`.*

```
> document=({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165})  
< {  
  name: 'Dunx',  
  loves: [ 'grape', 'watermelon' ],  
  weight: 704,  
  gender: 'm',  
  vampires: 165  
}
```

```
> db.unicorns.insert(document)
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0b0314dbc615f68cf6e604')
  }
}
```

```
> db.unicorns.find()
< {
  _id: ObjectId('6a0b028fdb615f68cf6e5f8'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
  _id: ObjectId('6a0b028fdb615f68cf6e5f9'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a0b028fdb615f68cf6e5fa'),
  name: 'Unicrom',
  loves: [
    'energon',
    'redbull'
  ],
  gender: 'm',
  vampires: 54
}
{
  _id: ObjectId('6a0b028fdb615f68cf6e602'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
{
  _id: ObjectId('6a0b0314dbc615f68cf6e604'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
```

Практическое задание 2.2.1:

1. Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.
2. Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций `findOne` и `limit`.

```
db.unicorns.find({gender: 'm'});
```

```
> db.unicorns.find({gender: 'm'});
< {
  _id: ObjectId('6a1028aff213ed3988fe88a4'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
  _id: ObjectId('6a1028aff213ed3988fe88a6'),
  name: 'Unicrom',
```

db.unicorns.find({gender: 'f'}).limit(3);

```
> db.unicorns.find({gender: 'f'}).limit(3);
< {
  _id: ObjectId('6a1028aff213ed3988fe88a5'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a1028aff213ed3988fe88a8'),
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80
}
```

```
{
  _id: ObjectId('6a1028aff213ed3988fe88a9'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 733,
  gender: 'f',
  vampires: 40
}
```

```
db.unicorns.find({gender: 'm'}).sort({name: 1});
```

```
> db.unicorns.find({gender: 'm'}).sort({name: 1});
< {
  _id: ObjectId('6a1028c5f213ed3988fe88af'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a1028aff213ed3988fe88a4'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
  _id: ObjectId('6a1028aff213ed3988fe88aa'),
  name: 'Kenny',
```

```
db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1);
```

```
> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1);
< {
  _id: ObjectId('6a1028aff213ed3988fe88a5'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

```
db.unicorns.findOne({gender: 'f', loves: 'carrot'});
```

```
> db.unicorns.findOne({gender: 'f', loves: 'carrot'});
< {
  _id: ObjectId('6a1028aff213ed3988fe88a5'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0});
```

```
> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0});
< {
  _id: ObjectId('6a1028aff213ed3988fe88a4'),
  name: 'Horny',
  weight: 600,
  vampires: 63
}
{
  _id: ObjectId('6a1028aff213ed3988fe88a6'),
  name: 'Unicrom',
  weight: 984,
  vampires: 182
}
{
  _id: ObjectId('6a1028aff213ed3988fe88a7'),
  name: 'Roooooodles',

```

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

```
db.unicorns.find().sort({ $natural: -1 });
```

```

> db.unicorns.find().sort({ $natural: -1 });
< {
  _id: ObjectId('6a1028c5f213ed3988fe88af'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a1028aff213ed3988fe88ae'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
{
  _id: ObjectId('6a1028aff213ed3988fe88ad'),
  name: 'Pilot',

```

Практическое задание 2.1.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
db.unicorns.find({}, {_id: 0, name: 1, loves: { $slice: 1 }});
```

```

> db.unicorns.find({}, {_id: 0, name: 1, loves: { $slice: 1 }});
< {
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{
  name: 'Aurora',
  loves: [
    'carrot'
  ]
}
{
  name: 'Unicrom',
  loves: [
    'energon'
  ]
}

```


Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
db.unicorns.find({weight: {$lt : 700}}, {_id: 0});
```

```
> db.unicorns.find({weight: {$lt : 700}}, {_id: 0});
< {
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
db.unicorns.find({gender: "m", weight: { $gte: 500 }, loves: { $all: ["grape", "lemon"] }}, {_id: 0});
```

```
> db.unicorns.find({gender: "m", weight: { $gte: 500 }, loves: { $all: ["grape", "lemon"] }}, {_id: 0});
< {
  name: 'Kenny',
  loves: [
    'grape',
    'lemon'
  ],
  weight: 690,
  gender: 'm',
  vampires: 39
}
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

```
db.unicorns.find({vampires: { $exists: false }});
```

```
> db.unicorns.find({vampires: { $exists: false }});  
< {  
  _id: ObjectId('6a1028aff213ed3988fe88ae'),  
  name: 'Nimue',  
  loves: [  
    'grape',  
    'carrot'  
  ],  
  weight: 540,  
  gender: 'f'  
}
```

Практическое задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
db.unicorns.find({gender: "m"}, {_id: 0, name: 1, loves: { $slice: 1  
}}).sort({name: 1});
```

```
> db.unicorns.find({gender: "m"}, {_id: 0, name: 1, loves: { $slice: 1 }}).sort({name: 1});  
< {  
  name: 'Dunx',  
  loves: [  
    'grape'  
  ]  
}  
{  
  name: 'Horny',  
  loves: [  
    'carrot'  
  ]  
}  
{  
  name: 'Kenny',
```

Практическое задание 3.1.1:

Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

```
db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0});
```

```
> db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0});
< {
  name: 'New York',
  mayor: {
    name: 'Michael Bloomberg',
    party: 'I'
  }
}
```

Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

```
db.towns.find({"mayor.party": { $exists: false }}, {name: 1, mayor: 1, _id: 0});
```

```
> db.towns.find({"mayor.party": { $exists: false }}, {name: 1, mayor: 1, _id: 0})
< {
  name: 'Punxsutawney ',
  mayor: {
    name: 'Jim Wehrle'
  }
}
```

Практическое задание 3.1.2:

Сформировать функцию для вывода списка самцов единорогов.

```
function showMaleUnicorns() {
  return db.unicorns.find({gender: "m"}, {_id: 0, name: 1});
}
```

Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.

```
var cursor = showMaleUnicorns().sort({name: 1}).limit(2); null;
```

Вывести результат, используя forEach.

```
cursor.forEach(function(obj) {  
  print(obj.name);  
});
```

```
> cursor.forEach(function(obj) {  
  print(obj.name);  
});  
< Dunx  
< Horny
```

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
db.unicorns.find({gender: "f", weight: { $lt: 600, $gte: 500 }}).count();
```

```
> db.unicorns.find({gender: "f", weight: { $lt: 600, $gte: 500 }}).count();  
< 2
```

Практическое задание 3.2.2:

Вывести список предпочтений.

```
db.unicorns.distinct("loves");
```

```
> db.unicorns.distinct("loves");  
< [  
  'apple',      'carrot',  
  'chocolate', 'energon',  
  'grape',      'lemon',  
  'papaya',     'redbull',  
  'strawberry', 'sugar',  
  'watermelon'  
]
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

```
db.unicorns.aggregate({$group: {_id: "$gender", count: {$sum: 1}}})
```

```
> db.unicorns.aggregate({$group: {_id: "$gender", count: {$sum: 1}}})  
< {  
  _id: 'f',  
  count: 5  
}  
{  
  _id: 'm',  
  count: 7  
}
```

Практическое задание 3.3.2:

Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.

```
db.unicorns.update({name: "Ayna", gender: "f"}, {$set: {weight: 800, vampires: 51}})
```

```
> db.unicorns.update({name: "Ayna", gender: "f"}, {$set: {weight: 800, vampires: 51}})
< DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
> db.unicorns.find({name: "Ayna"})
< {
  _id: ObjectId('6a1028aff213ed3988fe88a9'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 800,
  gender: 'f',
  vampires: 51
}
```

Практическое задание 3.3.3:

Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.

```
db.unicorns.update({name: "Raleigh", gender: "m"}, {$set: {loves: ["redbull"]}})
```

```

> db.unicorns.update({name: "Raleigh", gender: "m"}, {$set: {loves: ["redbull"]}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.unicorns.find({name: "Raleigh"})
< {
  _id: ObjectId('6a1028aff213ed3988fe88ab'),
  name: 'Raleigh',
  loves: [
    'redbull'
  ],
  weight: 421,
  gender: 'm',
  vampires: 2
}

```

Практическое задание 3.3.4:

Всем самцам единорогов увеличить количество убитых вампиров на 5.

```
db.unicorns.updateMany({gender: "m"}, {$inc: {vampires: 5}})
```

```

> db.unicorns.updateMany({gender: "m"}, {$inc: {vampires: 5}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 7,
  modifiedCount: 7,
  upsertedCount: 0
}
> db.unicorns.find({gender: "m"})
< {
  _id: ObjectId('6a1028aff213ed3988fe88a4'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 68
}
{
  _id: ObjectId('6a1028aff213ed3988fe88a6'),
  name: 'Unicrom',
  loves: [
    'energon',
    'redbull'
  ],
  weight: 984,
  gender: 'm',

```

Практическое задание 3.3.5:

Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.

```
db.towns.update({name: "Portland"}, {$unset: {"mayor.party": 1}})
```

```
{
  _id: ObjectId('6a0c4792410636c57ea64c4b'),
  name: 'Portland',
  populatiuon: 528000,
  last_sensus: 2009-07-20T00:00:00.000Z,
  famous_for: [
    'beer',
    'food'
  ],
  mayor: {
    name: 'Sam Adams'
  }
}
```

Практическое задание 3.3.6:

Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.

```
db.unicorns.update({ name: "Pilot", gender: "m" }, {$push: {loves: "chocolate"}})
```

```
< {
  _id: ObjectId('6a0b028fdb615f68cf6e601'),
  name: 'Pilot',
  loves: [
    'apple',
    'watermelon',
    'chocolate'
  ],
  weight: 650,
  gender: 'm',
  vampires: 59
}
```

Практическое задание 3.3.7:

Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.

```
db.unicorns.update({name: "Aurora", gender: "f"}, {$addToSet: {loves: {
$each: ["sugar", "lemon"] }}}})
```

```
< {
  _id: ObjectId('6a0b028fdb615f68cf6e5f9'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape',
    'sugar',
    'lemon'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

Практическое задание 3.4.1:

Удалите документы с беспартийными мэрами.

`db.towns.remove({"mayor.party": { $exists: false }})`

```
< {
  _id: ObjectId('6a0c60ac410636c57ea64c4e'),
  name: 'New York',
  popujatiuon: 22200000,
  last_sensus: 2009-07-31T00:00:00.000Z,
  famous_for: [
    'status of liberty',
    'food'
  ],
  mayor: {
    name: 'Michael Bloomberg',
    party: 'I'
  }
}
{
  _id: ObjectId('6a0c60ac410636c57ea64c4f'),
  name: 'Portland',
  popujatiuon: 528000,
  last_sensus: 2009-07-20T00:00:00.000Z,
  famous_for: [
    'beer',
    'food'
  ],
  mayor: {
    name: 'Sam Adams',
    party: 'D'
  }
}
```


Очистите коллекцию. Просмотрите список доступных коллекций.

db.towns.remove({})

```
> db.towns.find()
<
> show collections
< towns
  unicorns
  users
```

Практическое задание 4.1.1:

1. *Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.*
2. *Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.*
3. *Проверьте содержание коллекции единорогов.*

```
db.habitats.insertMany([
    { _id: "forest", fullname: "Enchanted Forest" },
    { _id: "mtn", fullname: "Raspberry Mountain" },
    { _id: "lake", fullname: "Silver Lake" }
])
```

```
db.unicorns.update({name: "Horny"}, {$set: {habitat: {$ref: "habitats", $id: "forest"}}})
```

```
db.unicorns.update({name: "Aurora"}, {$set: {habitat: {$ref: "habitats", $id: "lake"}}})
```

```
db.unicorns.update({name: "Kenny"}, {$set: {habitat: {$ref: "habitats", $id: "mtn"}}})
```

```

> db.unicorns.find()
< {
  _id: ObjectId('6a0dc853fbb6f4bbadb3e838'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63,
  habitat: DBRef('habitats', 'forest')
}
{
  _id: ObjectId('6a0dc853fbb6f4bbadb3e839'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43,
  habitat: DBRef('habitats', 'lake')
}
{
  _id: ObjectId('6a0dc853fbb6f4bbadb3e83e'),
  name: 'Kenny',
  loves: [
    'grape',
    'lemon'
  ],
  weight: 690,
  gender: 'm',
  vampires: 39,
  habitat: DBRef('habitats', 'mtn')
}

```

Практическое задание 4.2.1:

Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

```
db.unicorns.createIndex({name: 1}, {unique: true})
```

```

> db.unicorns.createIndex({name: 1}, {unique: true})
< name_1

```

Практическое задание 4.3.1:

1. *Получите информацию о всех индексах коллекции `unicorns`.*

2. Удалите все индексы, кроме индекса для идентификатора.
3. Попробуйте удалить индекс для идентификатора.

```
> db.unicorns.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_ ' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

```
> db.unicorns.dropIndexes()
< {
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
```

```
> db.unicorns.dropIndex({ _id: 1 })
✖ ► MongoServerError[InvalidOptions]: cannot drop _id index
```

Практическое задание 4.4.1:

1. Создайте объемную коллекцию *numbers*, задействовав курсор:
`for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}`
2. Выберите последних четыре документа.
3. Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра *executionTimeMillis*)
4. Создайте индекс для ключа *value*.
5. Получите информацию о всех индексах коллекции *numbers*.
6. Выполните запрос 2.
7. Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?
8. Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

```
db.numbers.find().sort({value: -1}).limit(4)
```

```
< {
  _id: ObjectId('6a0dcc32fbb6f4bbadb56ee6'),
  value: 99999
}
{
  _id: ObjectId('6a0dcc32fbb6f4bbadb56ee5'),
  value: 99998
}
{
  _id: ObjectId('6a0dcc32fbb6f4bbadb56ee4'),
  value: 99997
}
{
  _id: ObjectId('6a0dcc32fbb6f4bbadb56ee3'),
  value: 99996
}
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 110,
  totalKeysExamined: 0,
  totalDocsExamined: 100000,
```

db.numbers.createIndex({ value: 1 })

```
> db.numbers.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 25,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
```

Вывод

В ходе работы были освоены практические навыки работы с базой данных MongoDB. Были выполнены CRUD-операции для добавления, изменения, удаления и выборки документов, изучена работа с вложенными объектами и массивами в коллекциях. Также были использованы операции агрегации, выполнено создание и изменение ссылок между коллекциями, а также работа с индексами, включая анализ производительности запросов с индексами и без них с помощью `explain()`.